

Personal Portfolio & Skill Gap Tracker

Project documentation

1. Project Summary

Built a **React + TypeScript** web app for logging past projects, tagging the skills demonstrated in each one (with a depth level of Used, Implemented, or Architected), and comparing that project history against target job roles to surface **skill gaps**. The app now runs on a **proper Express + SQLite backend** that owns all data persistence and gap-analysis logic, plus an optional **Gemini AI** assist for writing project summaries and auto-classifying role, tech stack, and skills from free-form notes.

2. Tech Stack and Why

React + TypeScript + Vite → fast dev loop, type safety across the whole UI, and a component-driven interface for the project list, modals, and activity heatmap.

Tailwind CSS → utility-first styling for a dense, dashboard-style layout.

Express (Node.js) → REST API layer that stays in the same language as the frontend; in development it mounts Vite's middleware, in production it serves the built static assets.

better-sqlite3 (SQLite) → a synchronous, zero-config embedded database. No separate database server to run for a single-user personal tool, yet still a real relational schema with foreign keys, indexes, and cascading deletes.

Google Gemini API (@google/genai) → optional AI assist for generating project summaries and classifying role/tech/skills from raw notes, with a deterministic rule-based fallback when no API key is configured so the app is always usable offline.

3. Architecture Explanation

The project follows a genuine **three-tier architecture**. The React SPA talks to the backend only through REST endpoints under **/api** — it never touches `localStorage` or computes business logic itself anymore. An Express router (**server/routes.ts**) delegates each request to repository modules (**server/repositories/skills.ts**, **projects.ts**, **roles.ts**) that own all SQL against five SQLite tables: **skills**, **projects**, **project_skills** (join table with depth), **roles**, and **role_required_skills** (join table). A dedicated **server/gapAnalysis.ts** module computes readiness scoring — Covered / Partial / Missing status per required skill, plus a weighted coverage percentage — entirely server-side, so the client fetches ready-made results instead of duplicating that logic. Skills typed into a project or role are deduplicated case-insensitively into one shared pool via an **ensureSkill()** helper, keeping the master skill list consistent across projects and roles. After any create, edit, or delete, the client refetches projects, roles, skills, and gap analyses together so every view stays in sync with the database.

4. Bugs and Challenges

The project folder name itself contains an **"&"** character, which broke the auto-generated `npm/npmx .bin` shims on Windows — their `cmd/PowerShell` wrappers mis-parsed the path at the ampersand,

so **npm run dev** and **npx tsc** failed with `MODULE_NOT_FOUND`. Worked around it by invoking the underlying JS entry points directly through node (e.g. **node --import tsx server.ts**) instead of through the broken shims.

Migrating from client-only `localStorage` to a real backend meant every mutation (create, edit, delete a project or role) had to become asynchronous, where the UI previously wrote straight to `localStorage` inside a synchronous state update. Solved it by centralizing all reads and writes behind a small fetch-based API client (**src/data/api.ts**) and having the app refetch projects, roles, skills, and gap analyses together after every mutation, rather than trying to optimistically guess the server's resulting state.

Skills previously needed a client-generated id before a project or role could reference them.

Moving skill registration into the backend meant the client no longer creates skill ids at all — it just sends skill names and depth levels, and **ensureSkill()** handles dedup and insertion, which actually simplified the project and role form components.

5. What I Learned

Learned how to design a small relational schema from a set of TypeScript interfaces that used to be serialized straight into `localStorage`, including how to model a many-valued field (a project's tagged skills, a role's required skills) as a proper join table instead of a JSON blob column. Also learned to treat a derived value like gap-analysis coverage as a server-computed query rather than trusting the client to recompute it consistently in multiple places. Got hands-on with SQLite foreign keys and cascading deletes, WAL mode, and bundling a native Node addon (better-sqlite3) through esbuild for production, where **--packages=external** keeps native bindings out of the bundle so they still resolve correctly at runtime.

6. Why did you choose Express and SQLite?

Express, because the project already ran Node and TypeScript on both sides via Vite's dev middleware, so keeping the API in the same language let the Express app mount Vite's middleware in development and serve the static build in production without a second toolchain. SQLite (via better-sqlite3), because this is a single-user personal tool: there's no need for a hosted database server, the entire dataset lives in one portable **app.db** file, and better-sqlite3's synchronous API keeps the repository code simple — no `async/await` ceremony around every query — while still providing real foreign keys, indexes, and transactions.

7. What would you improve if you had more time?

Add authentication so the tracker supports more than one user — the schema already threads a **user_id** through every table, but the API currently hardcodes a single default user. Add pagination and server-side search for projects once a portfolio grows large. Move the Gemini calls behind a small cache so re-classifying similar notes doesn't re-hit the API every time. And add automated tests around the gap-analysis scoring logic and the repository layer, since that scoring is the core value proposition of the app.